Ethereum

• **Bitcoin is a decentralized application for money.**
• **Ethereum is a decentralized platform for *any* application.**

With **Bitcoin**, you can only write down debit and credit transactions: "Alice pays Bob 1 BTC."

- With **Ethereum**, you can write down debit/credit transactions, but you can *also* write down a set of rules in the notebook itself. For example: "If Alice pays Bob 1 ETH, and today is a Tuesday, then automatically send 0.1 ETH from Bob to Carol."

Ethereum Virtual Machine (EVM)

How can thousands of different computers, all over the world, run the same piece of code and be **100% certain** they all get the exact same result?

Think about your own computer. The result of a program can depend on the time of day, a random number, a file on your hard drive, or information you fetch from the internet. If every computer on the Ethereum network tried to run a program that said "fetch the price of gold from google.com," they might all get slightly different answers at slightly different times. The network would instantly fall out of consensus. The ledger would break.

The system needs to be perfectly **deterministic**. For a given input, the output must *always* be the same, no matter who runs it or when they run it.

Ethereum solves this problem with a brilliant concept: the **Ethereum Virtual Machine (EVM)**.

1. **It's a Specification, Not a Physical Thing:** The EVM is not a piece of hardware. It's a detailed blueprint for a CPU. It's a set of rules. Anyone can use this blueprint to build their own software version of the EVM (and they do, in different programming languages like Go, Rust, and Java). As long as everyone's software version follows the rules exactly, they are guaranteed to produce the same output.
2. **It's Completely Isolated (Sandboxed):** Code running inside the EVM has no access to the outside world. It cannot access the computer's network, its file system, or other processes. It lives in a "walled garden." The only information it can "see" is information that is already on the blockchain itself (like the current block number, account balances, or data stored by other contracts). This is the key to achieving determinism.
3. **It has a Simple Instruction Set:** Just like a real CPU has basic instructions like ADD, COPY, STORE, the EVM has its own set of simple instructions called **opcodes**. When a developer writes a program in a high-level language like Solidity (Ethereum's most popular language), it gets compiled down into these simple, universal EVM opcodes. This is what is actually stored on the blockchain.

No Randomness,No External API Calls and No Floating-Point Math

GAS Fees:

- **Programmer A** writes very clean, efficient code to calculate the average of 10 numbers.

- **Programmer B** writes messy, inefficient code to do the same task. It has unnecessary steps and loops.

Imagine the user of that program is a bit optimistic and thinks the code is more efficient than it is. They set the **Gas Limit** on their transaction to **50,000**. What happens to their transaction, and what happens to the fee they paid?

Two Types of Account in Ethereum

1. Externally Owned Accounts (EOAs)

- **What it is:** This is the account you and I would create with a wallet like MetaMask or Ledger.
- **How it's controlled:** It is controlled by a private key. Only the person with the private key can sign and authorize transactions from this account.
- **What it can do:** It can hold and send Ether (ETH). It can also initiate transactions to interact with other accounts (both EOAs and contracts).
- **Key Property:** An EOA **cannot** have code associated with it. It is passive. It only reacts when its owner tells it to do something.

2. Contract Accounts

- **What it is:** This is an account that is created when a developer deploys a smart contract to the network.
- **How it's controlled:** It is controlled by its own internal **code**. It has no private key. It "wakes up" and executes its code when it receives a transaction from an EOA or a message from another contract.

- **What it can do:** It can hold and send ETH. It can do anything its code tells it to do: run complex calculations, read/write its own storage, and even call other contract accounts.
- **Key Property:** A Contract Account is an autonomous agent living on the blockchain. It's a robot waiting for a command. This is where the code actually lives.

Key part of account

1. **Nonce:** A counter that goes up by one every time the account *sends* a transaction. This is a security feature to prevent someone from taking one of your old transactions and "replaying" it to send money again.
2. **Balance:** The amount of Ether (the native currency) this account owns, stored as a simple number.
3. **codeHash:** If this row represents a smart contract, this column contains the hash of its code. If it's a regular user account (an EOA), this column is empty. This is how the network distinguishes between users and code.
4. **storageRoot:** If this is a smart contract, it has its own little private database or "storage." This column contains the root hash of that storage, which is a fingerprint of all the data the contract is holding.

How to send Transaction

- `from: Your Address`
- `to: Your Friend's Address`
- `value: 1 ETH`
- `gasLimit: 21,000 (standard for a simple transfer)`
- `gasPrice: 20 Gwei`
- `nonce: 74 (your account's next transaction number, fetched from the network)`

Where the transaction is stored and where the balance is stored?

The **Ethereum state** refers to the current status of **all accounts and smart contracts** on the Ethereum blockchain at a specific point in time.

```
Old State + Transactions = New State
```

The "accounts" are not stored inside the blocks. They are stored in a separate, massive database on each node's hard drive. The block only contains a cryptographic *proof* (the stateRoot) that this database was updated correctly.

Database 1: The Blockchain (The Journal)

- **What it is:** A simple, linear list of blocks.
- **How it's stored:** This is literally a set of files on your hard drive, one after another (block_0, block_1, block_2, ...). Each block file contains its header and the list of all transactions included in it.
- **How it grows:** It only ever grows. You just keep appending new blocks to the end of the chain. It's an append-only log. This is relatively simple data storage.

Database 2: The State (The Current Balance Sheet)

- **What it is:** The massive "spreadsheet" of all accounts and their properties (nonce, balance, storage, code). This is a much more complex data structure.

- **How it's stored:** This is stored in a highly optimized key-value database on your node's hard drive (commonly using a database like LevelDB). The "key" is the account address, and the "value" is the account's data. This database is structured as a **Merkle Patricia Trie**.
    - It's a special kind of database that has one magical property: you can generate a single, 32-byte hash (the stateRoot) that acts as a unique fingerprint for the *entire contents of the database*. If even one bit changes in any account's balance anywhere, the final stateRoot hash will change completely.
- **How it changes:** This database is constantly being **modified**. When a new block arrives, your node reads the transactions, and for each one, it updates this state database. It might decrease one account's balance, increase another's, and change a contract's storage. It's a living, breathing database.

A node never blindly trusts its own local database. It constantly verifies its local database against the cryptographic commitments recorded on the blockchain.

- previousBlockHash: 00000000
- transactions: [Tx A]
- StateRoot
- Digital Signature of Validator
- Address of Validator
- Reward
- number

Rule of 2/3

Can a smart contract *start a transaction by itself*?

For example, could a Decentralized Finance (DeFi) lending protocol automatically send you your daily interest payment at exactly 8:00 AM every day, without anyone prompting it?

Smart Contract and a Token

Solidity Contract:

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.0;
```

```
contract SimpleCounter {
```

```
// State variable to store the counter value
```

```
uint256 public count;

// Constructor - runs once when contract is deployed

constructor() {
        count = 0;

// Initialize counter to 0

}

// Function to increment the counter by 1

function increment() public {
        count += 1;
    }

// Function to decrement the counter by 1

function decrement() public {
        require(count > 0, "Counter cannot go below zero");
        count -= 1;
    }

// Function to get current count (already public, so this is optional)

function getCount() public view returns (uint256) {
        return count;
    }

// Function to reset counter to zero

function reset() public {
        count = 0;
    }

// Function to set counter to specific value

function setCount(uint256 _newCount) public {
        count = _newCount;
    }
}
```

The Solution: The ERC-20 Token Standard

Instead of every developer inventing their own way to code a currency, the Ethereum community proposed a standard interface, a blueprint, called **ERC-20**

the contract has one primary variable in its storage: a

mapping

. You can think of this as a simple spreadsheet or a dictionary inside the contract itself.

mapping(address => uint256) public balances;

This line of code says: "Create a list where the key is an Ethereum address and the value is a number (the balance)."

- totalSupply(): A simple function that returns the total amount of the token that exists.
- balanceOf(address who): A function that looks up the balances mapping and returns the balance for the address who. (This answers Question 1).
- transfer(address to, uint256 amount): This is the key function. When you call it, the code does the following:
    1. Checks if your address (msg.sender) has enough tokens in the balances mapping.
    2. If yes, it subtracts the amount from your balance.
    3. It adds the amount to the to address's balance.
    4. If the check fails, it reverts the transaction.

ERC-20 is not a piece of software or a protocol. It is a technical *standard* or *specification*.

Think of it as a **blueprint** or a **universally agreed-upon set of rules** for creating a token on Ethereum.

- **Alice creates a token called "AliceCoin."** To check a balance, her function is named getAliceBalance(address). To send tokens, her function is sendAliceCoin(address, amount).
- **Bob creates "BobCoin."** His functions are fetch_balance(address) and move_tokens(address, amount).
- **Carol creates "CarolCoin."** Her functions are balanceOf(address) and do_transfer(address, amount).

The Solution: A Common Interface

The developers in the Ethereum community recognized this problem early on. A group of them wrote **E**thereum **R**equest for **C**omments **#20**. This document proposed a standard.

It said: "If you want to create a fungible token (where each token is identical, like a dollar bill), your smart contract **MUST** implement the following set of functions and events with these exact names and parameters."

ERC-20 is this mandatory public interface. It's a contract that a smart contract makes with the outside world.

Introduction To Solidity

Lesson 1: Dealing with simple variable

```
// 1. We tell the compiler what license we're using. It's just a good habit. // SPDX-License-Identifier: MIT // 2. We tell the compiler which version of Solidity to use. pragma solidity ^0.8.20; // 3. We define our contract. Think of it like a 'class' in C++. contract HelloWorld { // 4. This is a "State Variable". It is data stored permanently // with the contract on the blockchain. string public greeting = "Hello, World!"; }
```

Task:

1. Create a new contract named MyFavoriteNumber.
2. Inside this contract, create one state variable.
3. The variable should be named myNumber.
4. It should be a uint (unsigned integer, a positive number).
5. It should be public so we can easily read it.
6. Give it any initial value you want (e.g., uint public myNumber = 7;).

Lesson 2: Dealing with function

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; contract MyFavoriteNumber { uint public myNumber = 10; // This is our new function! function setMyNumber(uint _newNumber) public { myNumber = _newNumber; } }
```

Task:

1. Add a new public state variable to your contract. Make it a string and name it myStatus. Give it an initial value like "Learning Solidity".

2. Add a new public function called setMyStatus.
3. This function should take one string as input. just use the keyword calldata. So the input will look like string calldata _newStatus.
4. The function should update the myStatus state variable with the new input.

Lesson 3: **Functions That Read and Return Data**

We also have special keywords to tell Solidity that a function will **not** change any data. This is important because it means calling the function is **free**.

The two keywords are:

- **view**: Use this for a function that only **reads** state variables but doesn't change them.
- **pure**: Use this for a function that doesn't even read the state variables. It's a pure calculation that only depends on its inputs.

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; contract MyInfo
{ // I've renamed the contract to be more descriptive uint public
myNumber = 10; string public myStatus = "Learning Solidity"; function
setMyNumber(uint _newNumber) public { myNumber = _newNumber; } function
setMyStatus(string calldata _newStatus) public { myStatus = _newStatus;
} // This is our new VIEW function! function getInfo() public view
returns (string memory) { // NOTE: Combining strings in Solidity is a
bit strange. // We use a special function called abi.encodePacked. //
Don't worry about how it works for now. return
string(abi.encodePacked("My number is ", toString(myNumber), " and I am
currently ", myStatus)); } // This is a PURE helper function. It doesn't
read any state. // Its only job is to turn a number into a string.
function toString(string memory _i) internal pure returns (string
memory) { if (_i == 0) { return "0"; } uint j = _i; uint len; while (j
!= 0) { len++; j /= 10; } bytes memory bstr = new bytes(len); uint k =
len; while (_i != 0) { k = k-1; uint8 temp = (48 + uint8(_i - _i / 10 *
10)); bytes1 b1 = bytes1(temp); bstr[k] = b1; _i /= 10; } return
string(bstr); } }
```

Task:

1. Add a new function to your contract called doubleMyNumber.
2. It should take one uint as input (e.g., uint _numberToDouble).
3. It should be public.
4. It must be marked as pure, because it's just doing math and doesn't need to read myNumber or myStatus.
5. It must return a uint.
6. The logic inside the function should be simple: return _numberToDouble * 2;

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; contract
MathPractice { function sumOfEvenNumbers(uint _n) public pure returns
(uint) { uint total = 0; for (uint i = 1; i <= _n; i++) { if (i % 2 ==
0) { total = total + i; } } return total; } }
```

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; contract NumberGuesser { // This function takes a number as input and returns a string. function guess(uint _myGuess) public pure returns (string memory) { if (_myGuess == 7) { return "You guessed correctly!"; } else if (_myGuess < 7) { return "Too low, try again."; } else { return "Too high, try again."; } } }
```

1. The Foundation: byte vs. bytes

The core confusion often lies between a single unit and a collection.

- **byte**: This is an alias for bytes1. It represents a **single byte** of data (a number from 0 to 255). It is the fundamental atom of data.
    - *Analogy:* A single letter.
- **bytes**: This is a **dynamically-sized array of bytes**. It is a collection or sequence of bytes whose length can change.
    - *Analogy:* A word or a sentence, which is a collection of letters.

> First Principle: A bytes variable is simply a byte[].

2. Fixed Size vs. Dynamic Size: bytesN vs. bytes

This is about choosing the right tool for the job to save gas.

- **bytesN** (e.g., bytes4, bytes32): Use this when you know the **exact size** of your data beforehand. It is a fixed-size container.
    - *Analogy:* A rigid box designed to hold exactly N items.
    - *Benefit:* **Highly gas-efficient.** The compiler knows the size, making storage and retrieval cheaper.
    - *Use Case:* Storing hashes (bytes32), short IDs, or data with a known, unchanging structure.
- **bytes** (dynamic): Use this when the size of your data is **unknown or can change**.
    - *Analogy:* An expandable suitcase that adapts to its contents.
    - *Benefit:* **Flexible.**
    - *Use Case:* Storing user-provided text, digital signatures, or any data of variable length.

> Golden Rule: If the data can fit into 32 bytes and its size is fixed, always prefer bytesN over bytes or string to save gas.

3. Working with string vs. bytes

A string is a special kind of bytes array that is meant for human-readable UTF-8 text. It is less flexible on-chain.

- **To get the length of a string:** You must cast it to bytes first.
    - uint length = bytes(myString).length;
- **To join strings:** You cannot use +. You must use abi.encodePacked() and then cast the result back to a string.
    - string memory fullName = string(abi.encodePacked(firstName, " ", lastName));

4. Multi-Dimensional Structures: bytes[]

This represents a collection where each item is itself a dynamic collection.

- **bytes[]**: This is a **dynamic array of dynamic bytes arrays**.
    - *Analogy:* A library, where each book is a bytes variable (a collection of words/bytes), and the entire shelf of books is the bytes[] array.
    - *Use Case:* Storing a list of items where each item has a variable length, such as a list of user comments or a list of digital signatures.

5. Creating Dynamic Arrays in Memory

When you create a bytes or bytes[] array temporarily inside a function, you must use the new keyword and specify its size at that moment. The bytes on the left of the = is the type declaration, and its size is determined by the value on the right.

- bytes memory myData = new bytes(10); // Creates a temporary byte sequence of length 10.
- bytes memory myName = bytes("hello"); // Creates a temporary byte sequence of length 5.

Deep Dive on string and bytes

In many programming languages, strings are easy to work with. In Solidity, they are a bit more complex and less flexible due to the design of the EVM and the cost of gas. Understanding the difference between string and bytes is key to writing efficient code.

bytes: The Efficient Foundation

- **What it is:** A bytes variable is a dynamically-sized array of raw bytes. Think of it as byte[]. This is the EVM's "native" way of handling dynamic data.
- **Best for:** Raw, unstructured data that you don't necessarily need to read as human text. For example, a cryptographic signature, the bytecode of another contract, or any raw binary data.
- **Key Advantage:** bytes is more gas-efficient than string for on-chain operations. You have more direct control over it.

`You can get the length of a bytes array directly:`

```
bytes myData = "some data";
```

```
uint length = myData.length; // This works
```

string: The Human-Readable Layer

- **What it is:** A string variable is also a dynamically-sized byte array, but it's specifically encoded in **UTF-8**.
- **Best for:** Human-readable text that you expect to display on a website frontend. For example, a user's name, a status message, or an NFT's description.
- **Key Disadvantage:** Solidity does **not** provide many built-in tools to manipulate strings directly on-chain because it's very expensive.
    - You **cannot** get the length of a string with .length directly (myString.length does not work).
    - You **cannot** access characters by index directly (myString[0] does not work).
    - You **cannot** concatenate (join) two strings easily with + ("a" + "b" does not work).

```
string myStatus = "Learning Solidity"; // To get the length of the string: uint length = bytes(myStatus).length; // This returns 17 // To get the first character (as a byte): bytes1 firstCharAsByte = bytes(myStatus)[0]; // This gives the byte for 'L'
```

```
string name = "Rohit"; string status = "is learning"; // This is how you join them: string sentence = string(abi.encodePacked(name, " ", status, ".")); // The value of 'sentence' will be "Rohit is learning."
```

1. Dynamic-Sized Array of uint (Most Common)

This is an array that can grow or shrink. You use this when you don't know how many numbers you'll need to store in advance.

The Syntax:

You use empty square brackets [] after the type.

uint[] public myNumbers;

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; contract
DynamicUintArray { // This declares a dynamic array named 'myNumbers'.
// It is public, so we can see it. It starts empty. uint[] public
myNumbers; // A function to add a number to the end of the array.
function addNumber(uint _newNumber) public { // .push() adds the element
and increases the array's length by 1. myNumbers.push(_newNumber); } //
A function to get the current number of elements in the array. function
getCount() public view returns (uint) { return myNumbers.length; } // A
function to get a number at a specific position (index). function
getNumberAtIndex(uint _index) public view returns (uint) { return
myNumbers[_index]; } }
```

Fixed-Sized Array of uint

This is an array where the size is set when you write the code and can **never be changed**.

The Syntax:

You put the size inside the square brackets [size].

uint[5] public myFiveScores;

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; contract
FixedUintArray { // This declares a fixed-size array that can hold
EXACTLY 5 uints. uint[5] public myFiveScores; // We can initialize it
with values when we declare it. // The number of items must match the
size exactly. uint[3] public topThreeWinners = [101, 205, 117]; // A
function to change a value at a specific position. function
setScore(uint _index, uint _score) public { // We must make sure the
index is within our bounds (0 to 4). require(_index < 5, "Index out of
bounds."); myFiveScores[_index] = _score; } }
```

Lesson 4: **Adding Rules with require**

So far, anyone can call our functions and set any value they want. What if we want to add some rules? For example, what if we want to prevent someone from setting the myNumber to zero?

This is where require() comes in. It's one of the most important functions in Solidity.

require() checks if a condition is true.

- If the condition **is true**, the function continues.
- If the condition **is false**, the function stops immediately, reverses any changes it made, and gives back an error message.

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; contract
MyFavoriteNumber { uint public myNumber = 10; string public myStatus =
"Learning Solidity"; // Let's add the setMyNumber function back in for
this example function setMyNumber(uint _newNumber) public { // This is
our new rule! // It checks if the new number is greater than the current
one. require(_newNumber > myNumber, "New number must be greater than the
current one."); // This line will only run if the require() check
passes. myNumber = _newNumber; } function setMyStatus(string calldata
_myStatus) public { myStatus = _myStatus; } function doubleMyNumber(uint
_num) public pure returns (uint) { return _num * 2; } }
```

Task:

1. Modify the setMyStatus function.
2. Add a require() statement at the beginning of the function.
3. The rule should be: **The new status string cannot be empty.**
4. Give it a helpful error message, like "Status cannot be empty.".

Lesson 5: **msg.sender and Contract Ownership**

Every time someone calls a function on your contract, the Ethereum network provides some special information about the call. The most important piece of information is **who made the call**.

Solidity gives us a special, "magic" global variable for this: msg.sender.

- msg.sender: Is always equal to the **address** of the person or contract that is currently calling the function.

This allows us to create rules based on *who* is calling, not just *what* they're sending. This is how we implement **access control** or **ownership**.

The constructor: A Special Function

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; // This
contract has an owner! contract Owned { // 1. A state variable to store
the owner's address. address public owner; // 2. The constructor
function. // This runs automatically ONLY when the contract is deployed.
constructor() { // 3. We set the 'owner' to the address of the person
who deployed it. owner = msg.sender; } // A function that can only be
called by the owner. function doSomethingOnlyOwnerCanDo() public { // 4.
We require that the person calling this function IS the owner.
require(msg.sender == owner, "You are not the owner!"); // ... owner-
only logic would go here ... } }
```

Task:

1. Add a new public state variable of type address named owner.
2. Add a constructor to your contract. Inside the constructor, set the owner variable to msg.sender.
3. Modify the setMyNumber function. Add a **new require statement** at the very beginning of the function to check that msg.sender == owner. The error message should be something like "Only the owner can set the number.".

Lesson 6: Custom Modifiers

What if you have ten different functions that should only be callable by the owner? You would have to copy and paste that same require statement into all ten functions. This is repetitive and can lead to errors.

Solidity has a beautiful solution for this called a **modifier**.

A modifier is like a "wrapper" or a "pre-check" that you can attach to a function.

```
// 1. Define the modifier modifier onlyOwner() { require(msg.sender == owner, "Only the owner can call this function."); // 2. This is a special placeholder. It means: // "If the require check passed, run the rest of the function's code now." _; }
```

```
// Attach the modifier right after the visibility keyword function setMyNumber(uint _num) public onlyOwner { // The ownership check is now handled automatically by the modifier! // We can remove the require line from inside the function. require(_num > myNumber, "Your number must be greater than the current number"); myNumber = _num; }
```

Task:

1. In your MyFavoriteNumber contract, create a new modifier named onlyOwner.
2. The modifier should contain the require statement that checks if msg.sender == owner.
3. Remove the require statement from inside the setMyNumber function and add the onlyOwner modifier to its declaration instead.
4. For extra practice, let's say that anyone can *read* the status, but only the owner should be able to *change* it. Add the onlyOwner modifier to the setMyStatus function as well.

Lesson 7: Ether, Units, and payable Functions

Smart contracts can hold, receive, and send Ether, the native currency of the Ethereum blockchain. This is what makes them so powerful for finance, NFTs, and more.

Ether Units

First, it's important to know that while we talk about "ether," smart contracts do all their calculations in the smallest unit, called **wei**.

It's like how we have dollars and cents. You wouldn't do financial calculations in a mix of dollars and cents; you'd convert everything to cents first.

Here's the breakdown:

- 1 ether = 1,000,000,000,000,000,000 wei (1 with 18 zeros)
- 1 gwei = 1,000,000,000 wei (1 with 9 zeros) - *Gwei is often used to talk about gas prices.*

Solidity gives us convenient keywords for these units, so you don't have to type all the zeros.

- 1 ether
- 1 gwei
- 1 wei

So, 2 ether is the same as 2000000000000000000 wei. The keywords just make it readable.

Receiving Ether: The payable Keyword

By default, functions in a smart contract **cannot** accept Ether. If you try to send Ether to a normal function, the transaction will be rejected. This is a safety feature.

To make a function able to receive Ether, you must mark it as payable.

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; contract TipJar
{ // We can see how much Ether is stored in this contract function
getBalance() public view returns (uint) { // 'address(this).balance' is
a special property that // gives the Ether balance of the current
contract. return address(this).balance; } // This is a payable function.
Anyone can call it and send Ether. function sendTip() public payable {
// The function body can be empty! Its only job is to // receive the
Ether. } }
```

Task

1. Modify your setMyNumber function.
2. Make the function payable.
3. Add a **new rule** (require statement). This rule should check that the amount of Ether sent with the function call is **exactly 0.01 ether**.
    - **Hint 1:** Just like msg.sender, Solidity gives us another magic variable, msg.value, which holds the amount of wei sent with the call.
    - **Hint 2:** You can use the ether keyword for this. The condition will look like: msg.value == 0.01 ether.
4. Give it a good error message, like "You must pay exactly 0.01 ETH.".

Lesson 8: Sending Ether & Withdrawing Funds

A contract can hold Ether, but it can't "spend" it on its own. You need to write a function that explicitly tells the contract to send its balance to a specific address. This is a critical feature for any contract that handles money.

The Tool: .transfer()

The modern, safe, and recommended way to send Ether from a contract is by using the .transfer() method.

The syntax looks like this:

address_to_send_to.transfer(amount_to_send);

However, there's one small but important safety feature. A normal address variable cannot be sent Ether directly. You must first explicitly convert it to a special address payable type. This tells Solidity, "I am certain I want to send funds to this address."

The final, correct syntax is:

payable(address_to_send_to).transfer(amount_to_send);

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; contract TipJar
{ // 1. Add an address variable to store the owner. address public
owner; // 2. Use the constructor to set the owner to whoever deploys the
contract. constructor() { owner = msg.sender; } // 3. Create our
onlyOwner modifier to protect functions. modifier onlyOwner() {
require(msg.sender == owner, "You are not the owner."); _; } // Our
existing function to see the contract's balance. function getBalance()
public view returns (uint) { return address(this).balance; } // Our
existing function to receive tips. function sendTip() public payable {
// The Ether is automatically added to the contract's balance. } // 4.
Our NEW function to withdraw the funds. function withdraw() public
onlyOwner { // Get the amount of Ether this contract holds. uint amount
= address(this).balance; // The address of the owner who should receive
the funds. address ownerAddress = owner; // Transfer the entire amount
to the owner. // We MUST cast the owner's address to 'payable'.
payable(ownerAddress).transfer(amount); } }
```

How to send Money to some other address

```
function sendMoney(address _recipient, uint amount) public onlyOwner{
uint balancer = address(this).balance; require(balancer>= amount,
"Insufficient balance"); require(_recipient != address(0), "Cannot send
to the zero address."); payable(_recipient).transfer(amount); }
```

Create a vending machine of soda

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; /** *
@title VendingMachine * @dev A simple contract to simulate a
vending machine that sells "Digital Sodas". */ contract
VendingMachine { // The address of the person who owns the machine.
address public owner; // The number of sodas currently in the
machine. uint public sodaInventory; // Set the owner and initial
inventory when the contract is deployed. constructor() { owner =
msg.sender; sodaInventory = 100; // Start with 100 sodas. } // A
modifier to restrict certain functions to the owner only. modifier
onlyOwner() { require(msg.sender == owner, "Only the owner can call
this function."); _; } /** * @dev Allows anyone to buy a soda by
sending exactly 0.01 ETH. */ function buySoda() public payable { //
Check that there are sodas in stock. require(sodaInventory > 0,
"Sorry, out of stock!"); // Check that the user paid the exact
price. require(msg.value == 0.01 ether, "You must pay exactly 0.01
ETH per soda."); // If checks pass, decrease the inventory.
sodaInventory = sodaInventory - 1; } /** * @dev Allows the owner to
see the current Ether balance of the machine. */ function
getBalance() public view returns (uint) { return
address(this).balance; } /** * @dev Allows the owner to restock the
machine. * @param _amount The number of sodas to add to the
inventory. */ function restock(uint _amount) public onlyOwner {
sodaInventory = sodaInventory + _amount; } /** * @dev Allows the
owner to withdraw all collected profits. */ function
withdrawProfits() public onlyOwner { uint balance =
address(this).balance; require(balance > 0, "No profits to
withdraw."); // Transfer the entire balance to the owner's address.
payable(owner).transfer(balance); } }
```

Module 2:

Lesson 1: Mappings - The Contract's Memory

So far, our contracts can remember things about themselves (owner, sodaInventory). But they have no way to remember things about **specific users**. Our Vending Machine knows it sold 10 sodas, but it has no memory of *who* bought them. How can we store information tied to a specific person's address?

A mapping is a data structure that links a unique **Key** to a specific **Value**.

- Key -> Value
- Name -> Phone Number
- User's Address -> How many sodas they bought

The syntax for declaring this "phonebook" is:

mapping(KeyType => ValueType) public nameOfMapping;

- KeyType: The type of data you'll use to look up information. For user-specific data, this is almost always address.
- ValueType: The type of information you want to store. This could be a uint, a bool, or a string.

```
// This creates a "phonebook" where the key is an address // and the
value is a uint (the number of sodas bought). mapping(address => uint)
public sodasPurchased;
```

```
// This is the standard way to write this function without events.
function buySoda() public payable { require(msg.value == 0.01 ether,
"You must pay 0.01 ether"); require(soda > 0, "Sorry, out of stock!");
sodasPurchased[msg.sender]++; soda = soda - 1; }
```

Lesson 2: Structs - Creating Your Own Data Types

Right now, we are storing the number of sodas a person bought. But what if we wanted to store more information about each user? What if we wanted to store:

- How many sodas they bought.
- Their last purchase timestamp.
- Whether they are a "VIP" customer.
- List

We could create three separate mappings for this:

mapping(address => uint) public sodasPurchased;

mapping(address => uint) public lastPurchaseTime;

mapping(address => bool) public isVip;

```
struct NameOfStruct { Type1 variable1; Type2 variable2; // ... and so on
}
```

```
struct PurchaseInfo{ uint sodasBought; bool vip; uint lastPurchase; }
mapping (address => PurchaseInfo) public sodaPurchased;
```

```
function buySoda() public payable { require(msg.value == 0.01 ether,
"You must pay 0.01 ether"); require(soda > 0, "Sorry, out of stock!");
PurchaseInfo storage customer = sodaPurchased[msg.sender];
customer.sodasBought++; if(customer.sodasBought==5) { customer.vip =
true; } customer.lastPurchase = block.timestamp; soda = soda - 1; }
```

Lesson 3: Visibility keywords control who is allowed to call your functions or access your state variables.

1. public

- **Who can access it? Anyone and anything.**
  - External users (people with wallets).
  - Other smart contracts.
  - Functions within the same contract.
  - Functions in contracts that inherit from this one.
- **Analogy:** A public website page. Anyone can visit it.

```
uint public myNumber; // Anyone can read this via the auto-generated
'myNumber()' function. function buySoda() public payable { ... } //
Anyone can call this.
```

2. external

- **Who can access it? Only external users or other smart contracts.**
  - You **cannot** call an external function from another function *inside the same contract*

```
// Only outside users/contracts can call this. // Calling it from
'someOtherFunction' below would fail. function deposit() external
payable { ... } function someOtherFunction() public { // deposit(); //
THIS WOULD CAUSE A COMPILE ERROR // this.deposit(); // You could do
this, but it's inefficient and bad practice. }
```

3. internal

- **Who can access it?**
  - Only functions **within the same contract**.
  - And functions in contracts that **inherit** from this one (its "children").
- **Analogy:** A "family recipe". It's known to everyone in the immediate family (the contract) and can be passed down to the children (derived contracts), but it's not shared with the general public.

```
function _calculateBonus(uint _sales) internal pure returns (uint) {
return _sales / 10; } function paySalary(uint _sales) public { uint
bonus = _calculateBonus(_sales); // Calling the internal helper
function. // ... }
```

4. private

- **Who can access it?**
  - Only functions **within the very same contract** where it is defined.
  - It is **not** available to contracts that inherit from this one.

```
contract Base { uint private mySecretNumber = 42; function
revealSecret() public view returns (uint) { return mySecretNumber; //
This is allowed. } } contract Derived is Base { // 'Derived' inherits
from 'Base' function tryToAccessSecret() public view returns (uint) { //
return mySecretNumber; // THIS WOULD CAUSE A COMPILE ERROR. The secret
is private to 'Base'. } }
```

Summary Table

Visibility	From Outside Contract?	From Same Contract?	From Derived Contract?	Getter F
public	✅ Yes	✅ Yes	✅ Yes	✅ Yes
external	✅ Yes	❌ No	No❌	N/A (not
internal	❌ No	✅ Yes	✅ Yes	❌ No
private	❌ No	✅ Yes	❌ No	❌ No

Lesson 4: Interacting with Other Contracts

Real-world applications on Ethereum are rarely built as a single, monolithic contract. They are often a system of multiple, smaller contracts that work together. This is a core principle called **composability**.

How can our VendingMachine contract call a function on a completely separate PriceOracle contract to get the latest price for a soda?

The First Principle: A Contract Needs a "Menu" to Order From Another

To call a function on another contract, your contract needs to know two things:

1. **Where** the other contract is (its address).
2. **What** functions are available on it (its "menu").

This "menu" is called an **interface** in Solidity.

An interface is a bare-bones contract definition. It's like a C++ header file. It lists the functions you can call but contains none of the actual logic.

```
interface IoraclePrice { function getPrice() external view
returns(uint); }
```

Use the Interface in Your Contract

```
contract vendingMachine{ // A variable that will point to the real
sodaVendor contract. IoraclePrice public priceOracle; }
```

Next, when we deploy our VendingMachine, we need to tell it the address of the real PriceOracle contract. The constructor is the perfect place for this.

```
constructor(address _addressOracle){ priceOracle =
IoraclePrice(_addressOracle); }
```

Finally, we can now call the function from the other contract as if it were part of our own!

```
uint _price = priceOracle.getPrice();
```

Final Code

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; import
"./IoraclePrice.sol"; contract sodaVendor is IoraclePrice{ address
public owner; uint public price; modifier onlyOwner{
require(msg.sender==owner,"You are not the owner"); _; } constructor(){
owner = msg.sender; price = 1 ether; } function getPrice() external view
returns(uint) { return price; } function setPrice(uint _price) public
onlyOwner { price = _price; } }
```

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; import
"./IoraclePrice.sol"; contract vendingMachine{ uint soda; address public
owner; IoraclePrice public priceOracle; constructor(address
_addressOracle){ soda = 100; owner = msg.sender; priceOracle =
IoraclePrice(_addressOracle); } function getBalance() public view
returns(uint){ return address(this).balance; } modifier onlyOwner(){
require(msg.sender == owner, "You are not allowed"); _; } function
buySoda() public payable { uint _price = priceOracle.getPrice();
require(msg.value == _price , "You must pay 0.01 ether"); require(soda >
0, "Sorry, out of stock!"); soda = soda - 1; } function addStock(uint
_soda) public onlyOwner{ soda = soda + _soda; } function withdraw()
public onlyOwner{ payable(msg.sender).transfer(address(this).balance); }
}
```

Lesson 05: Events - The Voice of Your Contract

Imagine a user calls your buySoda() function. The transaction is sent, it gets mined 12 seconds later, and the state of the sodaInventory is updated on the blockchain.

But how does the user's **website** or **wallet** know that this happened?

- How can the website update its display to say "Purchase Successful!"?
- How can an analytics site know when a sale occurred and who the buyer was?

A function that changes the state cannot return a value to the outside world. So we need another way for the contract to "shout" to the world, "Hey, something important just happened!"

The Solidity Keywords: event and emit

Step 1: Define the Event (Create the Log Entry Template)

```
// This defines a template for a log entry called
'OwnershipTransferred'. // It will have two columns: 'previousOwner' and
'newOwner'. event OwnershipTransferred(address previousOwner, address
newOwner); // Another example: event SodaSold(address buyer, uint
quantity);
```

Step 2: Emit the Event (Write in the Log Book)

```
// Inside the transferOwnership function in our Ownable contract:
function transferOwnership(address _newOwner) public onlyOwner { // ...
require checks ... address oldOwner = owner; // Get the old owner's
address owner = _newOwner; // Update the state // Now, write the log
entry. // This SHOUTS to the outside world what just happened. emit
OwnershipTransferred(oldOwner, _newOwner); }
```

```
event OwnershipTransferred(address indexed previousOwner, address
indexed newOwner); //The indexed keyword is an optimization. It tells
the blockchain to make that specific "column" of your log book
searchable. It's like putting a tab or an index on that topic.
```

The problem is, a single "DELIVERED" message is not enough information to build a rich user interface.

Let's use our VendingMachine and an imaginary website that interacts with it.

The Scenario: A "Silent" but Successful Transaction

Imagine our buySoda function has **no event**.

1. A user, Alice, is on our CoolSodas.com website. She sees "Inventory: 100".
2. She clicks "Buy Soda for 0.01 ETH".
3. MetaMask pops up, she confirms, and sends the transaction.
4. 12 seconds later, MetaMask gives her a "Success!" notification. Her personal wallet now shows -0.01 ETH.

Now, what happens on the website? The website only knows that *some transaction* succeeded. It faces several problems:

- **Problem #1: What was the result?** Did the purchase actually go through? What if the contract had a buySnack() function that also cost 0.01 ETH? The website can't tell the difference just from a "Success" message. How does it know to update the "Your Sodas" counter for Alice?
- **Problem #2: What data changed?** The website's display still says "Inventory: 100". How does it know the new inventory is 99? It would have to make a *separate, new view call* to the sodaInventory() function to get the updated value. This is slow and inefficient.
- **Problem #3: How to build a history?** Alice wants to see a list of her past purchases on her account page. How does the website build this list? It would have to scan the entire history of the blockchain, look at every single transaction Alice has ever made, and try to guess which ones were for buying sodas from our contract. This is incredibly slow and practically impossible.

How Events Solve ALL These Problems

Now, imagine our buySoda function **does** emit SodaSold(msg.sender, 1);.

1. Alice clicks "Buy Soda."
2. Her transaction succeeds 12 seconds later.
3. Our website's JavaScript code has been **listening** for SodaSold events. It immediately sees a new event: { event: "SodaSold", args: { buyer: "Alice's_Address", quantity: 1 } }.

Now, the website has all the information it needs, instantly and without any guesswork:

- **Solves Problem #1:** The website sees the SodaSold event name and knows **exactly** what happened. It can confidently show a "Soda Purchase Successful!" popup and increment Alice's personal soda counter on the screen.
- **Solves Problem #2:** A well-designed event can also include the new state. For example, event SodaSold(address buyer, uint quantity, uint newTotalInventory);. By including the new inventory count in the event itself, the website doesn't need to make a second call. It can update the "Inventory" display directly from the event data.

- **Solves Problem #3:** To build Alice's purchase history, the website makes a single, very fast query to the blockchain: "Please give me all past SodaSold events where the buyer was Alice's address." The blockchain returns a clean list of all her past purchases instantly. This is only possible because the buyer address was indexed.

Conclusion:

A "Success" message from your wallet only tells you that your transaction didn't error out.

An **event** tells your application:

- **WHAT** specific action happened (SodaSold).
- **WHO** it happened to (buyer: Alice).
- **WITH WHAT DATA** (quantity: 1).

Lesson 06: Inheritance - Building on the Shoulders of Giants

The Problem We Need to Solve

Look at the two contracts you just wrote: sodaVendor and vendingMachine. What code do they have in common?

Both of them have this identical block of code:

```
address public owner; modifier onlyOwner { ... } constructor() { owner = msg.sender; }
```

The First Principle: Create Reusable Building Blocks

The professional solution is to extract this common, reusable logic into its own "base" or "parent" contract. Then, other contracts can simply **inherit** this functionality, like a child inheriting traits from a parent.

This is called **Inheritance**.

The Solidity Keyword: is

Step 1: Create the Base Ownable.sol Contract

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; contract
Ownable{ address public owner; constructor(){ owner = msg.sender; }
event OwnerShipTransfer(address indexed previousOwner,address indexed
newOwner); modifier onlyOwner(){ require(msg.sender==owner,"You are not
the owner"); _; } function transferOwnerShip(address _newOwner) public
virtual onlyOwner{ require(_newOwner!=address(0),"Invalid address");
emit OwnerShipTransfer(owner, _newOwner); owner = _newOwner; } }
```

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; import
"./IsodaPrice.sol"; import "./Ownable.sol"; contract sodaVendor is
IsodaPrice, Ownable{ uint public price; constructor(){ price = 1 ether;
} function getPrice() external view returns(uint) { return price; }
function setPrice(uint _price) public onlyOwner { price = _price; } }
```

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; import
"./IsodaPrice.sol"; import "./Ownable.sol"; contract vendingMachine is
Ownable{ uint soda; IsodaPrice public priceOracle; event
SodaPurchase(address buyer, uint quantity); constructor(address
_addressOracle){ soda = 100; priceOracle = IsodaPrice(_addressOracle); }
function getBalance() public view returns(uint){ return
address(this).balance; } function buySoda() public payable { uint _price
= priceOracle.getPrice(); require(msg.value == _price , "You must pay
0.01 ether"); require(soda > 0, "Sorry, out of stock!"); soda = soda -
1; emit SodaPurchase(msg.sender,1); } function addStock(uint _soda)
public onlyOwner{ soda = soda + _soda; } function withdraw() public
onlyOwner{ payable(msg.sender).transfer(address(this).balance); } }
```

Module 3: Automated Testing & Security

The Problem We Need to Solve:

How can we be confident that our VendingMachine works perfectly under all conditions?

- How do we prove that buySoda correctly decreases the inventory?
- How do we prove that if a user pays the wrong amount, the transaction correctly fails?
- How do we prove that only the owner can call withdraw?

Manually testing all of this is slow and error-prone. We need a way to write a script that tests all of these conditions automatically, every time we make a change to our code.

The First Principle: A Test is a "Simulation" of Your Contract

An automated test is a script (usually written in JavaScript or TypeScript) that "pretends" to be a user of your contract. It will:

1. Deploy a fresh copy of your contract to a temporary, local blockchain.
2. Call your functions with specific inputs.
3. Check the results to see if they match what you expected.

The Tool: Hardhat

To do this, we must finally move to a professional development environment. The industry standard is **Hardhat**. Hardhat is a command-line tool that gives you everything you need:

- A structured project folder.
- A way to compile your contracts.
- A built-in local Ethereum network for instant testing.
- A powerful testing framework to write and run your automated tests.

Initialize Your Project with hardhat init

Create and Enter Your Project Folder:

```
mkdir my-ignition-project cd my-ignition-project
```

Initialize the project

```
npm init -y npm install --save-dev hardhat npx hardhat init
```

Compile with Hardhat

```
npx hardhat compile
```

Run the Sample Test

```
npx hardhat test
```

1. contracts/

- **What It Is:** This is the **heart of your project**. It's the folder where you write and store all of your Solidity source code files (the .sol files).
- **Purpose:** To contain the raw logic and blueprints for your smart contracts.
- **Analogy:** This is the "kitchen" where you write your recipes. It contains the recipes for Ownable, IPriceOracle, sodaVendor, and vendingMachine.
- **How Hardhat Uses It:** When you run npx hardhat compile, Hardhat looks exclusively inside this folder, finds every .sol file, and compiles them into bytecode and ABIs.

- **Key Content:**
  - Your main application contracts (vendingMachine.sol).
  - Any base contracts you are inheriting from (Ownable.sol).
  - Any interfaces you are using (IPriceOracle.sol).

---

2. ignition/modules/

- **What It Is:** This folder holds your **deployment scripts**, written as TypeScript or JavaScript "modules." This is a newer feature of Hardhat, replacing the older scripts/ folder for deployments.
- **Purpose:** To define a reliable, repeatable, and clear plan for how to deploy your system of contracts to a blockchain. A deployment is often more complex than just deploying one contract; you have to deploy them in the right order and link them together.
- **Analogy:** This is the "Catering Plan" for a big event. It doesn't just list the recipes; it gives the step-by-step instructions for the chefs:
  1. "First, bake the sodaVendor cake."
  2. "Then, take the sodaVendor cake and use it to build the vendingMachine wedding cake."
- **How Hardhat Uses It:** When you run npx hardhat ignition deploy ./ignition/modules/YourDeployFile.ts, Hardhat executes the plan defined in that file. It manages the dependencies between contracts, ensuring sodaVendor is deployed before vendingMachine which needs its address.
- **Key Content:**
  - TypeScript (.ts) or JavaScript (.js) files that use the buildModule function from Hardhat Ignition to define the deployment sequence.

---

3. test/

- **What It Is:** This folder contains all of your **automated tests**. These are also written in TypeScript or JavaScript.
- **Purpose:** To prove that your contract logic is correct. You write scripts that simulate user actions and check if the results match your expectations. This is your most important safety net.

- **Analogy:** This is the "Food Safety and Quality Assurance Lab." Before any food leaves the kitchen, it goes to the lab. The lab runs tests:
    - "Test 1: Does the buySoda function correctly reduce inventory by 1? -> PASS"
    - "Test 2: If a user pays the wrong amount, does the transaction fail correctly? -> PASS"
    - "Test 3: If someone other than the owner tries to withdraw, does it fail? -> PASS"
- **How Hardhat Uses It:** When you run npx hardhat test, Hardhat finds every file in this folder ending in .test.ts or .test.js. For each test, it spins up a fresh, temporary local blockchain, deploys your contracts (often using your Ignition module), runs the test steps, and reports if they passed or failed.
- **Key Content:**
    - Test files that use frameworks like Mocha (describe, it) and Chai (expect) to structure the tests and make assertions.

After Compilation we will get two files

1. artifacts/

- **What It Is:** The "artifacts" folder contains the **final output** of the compilation process. It holds the "built" versions of your contracts, ready to be used.
- **Purpose:** To store the two most important pieces of information for each compiled contract: its **Bytecode** and its **ABI**.

2. cache/

- **What It Is:** This is a **temporary storage folder** used by Hardhat to make your development process faster.
- **Purpose:** To store information about your compiled files so that Hardhat doesn't have to recompile everything from scratch every single time.

```
// File: ignition/modules/Deploy.js const { buildModule } =
require("@nomicfoundation/hardhat-ignition/modules"); const { parseEther
} = require("ethers"); // We define a module that describes the
deployment of our contracts. module.exports =
buildModule("VendingModule", (m) => { // --- Deployment of the Price
Oracle --- // We need to set an initial price when deploying the
sodaVendor. // Let's set it to 0.01 ether. The `parseEther` function
from ethers.js // is the standard way to convert a string like "0.01"
into wei. const initialPrice = parseEther("0.01"); // The 'm.contract()'
function tells Ignition to deploy a contract. // First, we deploy our
'sodaVendor' contract. // The second argument is an array of constructor
arguments. const sodaVendor = m.contract("sodaVendor", []); // Hardhat
Ignition is still evolving, and setting the price // via a separate call
is a common pattern for now. // We can add calls to our contracts after
deployment. m.call(sodaVendor, "setPrice", [initialPrice]); // ---
Deployment of the Vending Machine --- // The vendingMachine's
constructor needs the address of the sodaVendor. // Ignition is smart.
When we pass the 'sodaVendor' contract object // as an argument here,
Ignition knows it needs to wait for sodaVendor // to be deployed first
and then pass its address. const vendingMachine =
m.contract("vendingMachine", [sodaVendor]); // The module returns an
object with the deployed contracts, // so we can easily access them
later in tests or other scripts. return { sodaVendor, vendingMachine };
});
```

Automated Testing - Gaining Confidence in Your Code

We have written a VendingMachine contract. We think it works. We've clicked around in Remix, and it seems okay. But how can we be **sure**?

- What if we make a small change later and accidentally break the buySoda function?
- What if there's a rare edge case we didn't think of?
- What if we're building a system that will handle millions of dollars? "Seems okay" is not good enough.

Manually clicking through every possible scenario every time you make a change is slow, boring, and impossible to do perfectly. You will miss things.

```
const { expect } = require("chai"); describe("Testing the math function", function(){ it("Should correctly add Two Number", function(){ let a = 10; let b = 20; const expectedResult = 30; const result = a+b; expect(result).to.equal(expectedResult); }); it("Should correctly subtract Two Number", function(){ let num1 = 20; let num2 = 10; const expectedResult = 10; const result = num1-num2; expect(result).to.equal(expectedResult); }); it("Should give incorrect output of 2 number", function(){ let num1 = 10; let num2 = 20; const expectedResult = 5; const result = 10-20; expect(result).to.equal(expectedResult); }) });
```

run the code

```
npx hardhat test
```

Testing Our Vending Machine:

The Goal: To test just one single thing: **Does the owner get set correctly when we deploy our contract?**

1. How do we tell our test script to deploy our contracts?
2. How do we get the address of the owner who deployed it?
3. How do we then "ask" the deployed contract who its owner is?

A Test Needs a "Setup Phase"

Before we can test anything, we need to set up the world. For us, "setting up" means deploying our contracts to the temporary Hardhat Network. This setup process is called a **fixture**.

Step 1: Create the Test File and Import Tools

```
// This function will help us run our setup efficiently. const {
loadFixture } = require("@nomicfoundation/hardhat-toolbox/network-
helpers"); // This is our assertion tool. const { expect } =
require("chai"); // These are the core Hardhat tools to talk to the
blockchain. const { ethers, ignition } = require("hardhat"); // This is
our own deployment plan that we wrote earlier. const VendingModule =
require("../ignition/modules/Deploy.js");
```

Step 2: Define the describe Block

Every test file needs a main container.

```
describe("VendingMachine Deployment", function () { // Our setup and
tests for deployment will go here. });
```

Step 3: Define the Fixture - Our "Setup" Function

Inside the describe block, we will write an async function. Its only job is to deploy our contracts. We will call it deployVendingFixture.

The Hardhat Network gives us 20 free test accounts. We need to get a handle on them. The first account is, by default, the one that does the deploying.

Now we need to run our deployment script. We do this with ignition.deploy() and we pass it the module we imported.

The fixture needs to give the results of its setup (the accounts and the deployed contracts) to the actual tests. We do this with a return statement.

```
async function deployVendingFixture() { const [owner] = await
ethers.getSigners(); const { vendingMachine } = await
ignition.deploy(VendingModule); // We return an object containing
everything a test might need. return { vendingMachine, owner }; }
```

Step 4: Write the Test (it block) Using the Fixture

```
it("Should Correctly set the Deployer as owner", async function(){ const
{ vendingMachine, owner } = await loadFixture(deployVendingFixture);
const deployedOwnerAddress = await vendingMachine.owner();
expect(deployedOwnerAddress).to.equal(owner.address); })
```

The Benefit of loadFixture: Speed Through Snapshots

Here's exactly what loadFixture does, which is even smarter than just "deploying it only once."

1. **The First Time:** The very first time an it block calls loadFixture(deployVendingFixture), Hardhat does the following:
    - It runs your deployVendingFixture function completely. This involves sending transactions, deploying contracts, etc. This is the "slow" part.
    - After the fixture is done, Hardhat takes a **"snapshot"** of the entire state of the blockchain. It's like taking a full backup or a photograph of the blockchain at that exact moment.
2. **For Every Subsequent Test:** The next time an it block calls loadFixture(deployVendingFixture), Hardhat does **NOT** re-deploy the contracts. Instead, it does something much faster:
    - It instantly **resets** the blockchain back to the clean "snapshot" it took after the first run.
    - It then returns the same deployment results (the contract objects, accounts, etc.) from that original setup.

Why is this better than just deploying once?

This is the key to the principle of **Test Isolation**.

Let's imagine you have two tests:

- it("Test A: Should let a user buy a soda")

- it("Test B: Should show an inventory of 100 on deployment")

Without loadFixture (the slow, bad way):

If you deployed once and shared the *same* contract instance between all tests, you would have a huge problem.

1. Test A runs. It buys a soda. The inventory of the *shared contract* is now 99.
2. Test B runs. It checks the inventory of the *same shared contract*. It gets 99.
3. **Test B now FAILS!** It expected 100, but Test A modified the state. The tests are interfering with each other. This is a testing nightmare.

With loadFixture (the fast, professional way):

1. The fixture runs for the first time before Test B. It deploys the contracts. The inventory is 100. A snapshot is taken.
2. Test B runs. It calls loadFixture, which returns the clean state. It checks the inventory. It sees 100. **Test B PASSES.**
3. Before Test A runs, Hardhat **instantly resets the blockchain** to the clean snapshot. The inventory is magically back to 100.
4. Test A runs. It calls loadFixture, gets the clean state. It buys a soda. The inventory becomes 99. It checks its own logic. **Test A PASSES.**

Testing Failure Cases and require Statements

The Goal: To prove that our contract correctly rejects transactions that violate its rules. Specifically, we will test that a user cannot buy a soda if they send the wrong amount of Ether.

It's not enough to know that a transaction failed. We need to know that it failed for the **exact reason** we expect. Did it fail because the payment was wrong, or did it fail because of some other random bug?

The Tool: Chai's revertedWith Matcher

The syntax looks like this:

await expect(some_transaction).to.be.revertedWith("Error message");

This powerful one-liner does three things:

1. It executes the some_transaction.
2. It **asserts** that the transaction **must fail** by reverting.
3. It **asserts** that the error message returned by the revert **exactly matches** the string you provide.

```
it("Should Revert if some Payment is failed", async function(){ const
{vendingMachine,owner:buyer} = await loadFixture(deployVendingFixture); (
await
expect(vendingMachine.connect(buyer).buySoda({value:price})).to.be.reverI
Payment amount for Soda"); })
```

Task: Write a new it block that tests for **access control**.

1. The description should be something like "Should prevent non-owners from withdrawing funds".
2. Use the deployVendingFixture to set up the contract.
3. Have the buyer account (who is not the owner) attempt to call the withdraw() function.

4. Use await expect(...).to.be.revertedWith(...) to assert that this transaction fails.

```
it("Should prevent non-owners from withdrawing funds", async function ()
{ const {vendingMachine,buyer} = await
loadFixture(deployVendingFixture); await
expect(vendingMachine.connect(buyer).withdraw()).to.be.revertedWith("You
are not a Owner"); })
```

- **Test Case #4: Out of Stock**
  - **Goal:** Prove that a user cannot buy a soda if the inventory is zero.
  - **Arrange:** Deploy the contract. You'll need to simulate buying all 100 sodas to empty the inventory. A for loop is perfect for this. Or, even better, have the owner set the inventory to 0. Let's add a setInventory function to make this easier.
  - **Act:** Have a user try to call buySoda().
  - **Assert:** Expect the transaction to.be.revertedWith("Sorry, out of stock!").

```
it("Should Prevent Buying Soda if stock is zero", async function() { cons
{vendingMachine,buyer,sodaVendor} = await loadFixture(deployVendingFixtur
price = await sodaVendor.getPrice(); for(let i=0;i<100;i++){ await
vendingMachine.connect(buyer).buySoda({value:price}) } expect(await
vendingMachine.soda()).to.equal(0); await
expect(vendingMachine.connect(buyer).buySoda({value:price})).to.be.reverti
out of stock!"); })
```

Important Note for writing await, whether to write it inside the expect or write it outside

The First Principle: await Waits for a Promise to Finish

In async JavaScript, the await keyword can only be used in front of something that returns a **Promise**. A Promise is an object that represents a future result of an asynchronous operation.

1. **Calling a contract function ALWAYS returns a Promise.**
    - vendingMachine.soda() -> This returns a Promise<BigNumber> (a promise of the inventory number).
    - vendingMachine.buySoda() -> This returns a Promise<TransactionResponse> (a promise of the transaction being sent).

Scenario 1: You are asserting a VALUE that you have already received.

This happens when you test view functions or state variables.

```
// Test: "The inventory should be 100." // 1. Get the value. This is an async operation, so we must await it. const inventory = await vendingMachine.soda(); // At this point, 'inventory' is no longer a Promise. It is the actual number, 100. // 2. Assert the value. 'expect()' itself is NOT async. It's just a regular function. // We are checking the concrete value we already have. expect(inventory).to.equal(100);
```

Scenario 2: You are asserting the BEHAVIOR of a TRANSACTION.

This happens when you test for reverts (failures). You are not checking a returned value. You are checking **what happens to the transaction itself**.

The expect() function from hardhat-chai-matchers has been specially modified for this.

1. vendingMachine.buySoda(...) is called. It immediately returns a Promise of a transaction.
2. You pass this **unresolved Promise** directly into expect().
3. The special revertedWith matcher then takes over. It says, "Okay, I have a promise of a transaction. I will now await it myself, *inside a try...catch block*."
4. It waits for the transaction to complete.
    - If the transaction succeeds, the matcher knows the expectation was wrong and throws an error: "Expected transaction to be reverted, but it didn't."
    - If the transaction fails (reverts), the matcher inspects the error message. If the message matches, the test passes. If it doesn't match, it throws a different error: "Expected revert reason 'X', but got 'Y'."
5. The await at the very beginning of the line is waiting for this entire expect(...) process to complete.

Test Case #5: Successful Withdrawal

- **Goal:** Prove that the owner can successfully withdraw the contract's entire balance.

- **Arrange:**
    1. Deploy the contract.
    2. Have a buyer purchase one soda, sending 0.01 ETH to the contract. The contract's balance is now 0.01 ETH.
    3. Get the owner's ETH balance *before* the withdrawal.
- **Act:** Have the owner account call the withdraw() function.
- **Assert:**
    1. Check that the contract's new balance is 0.
    2. Check that the owner's ETH balance has **increased** by approximately 0.01 ETH. (It won't be exact due to gas fees, so Chai has special matchers like to.changeEtherBalance).

```
it("Proving Owner withdraw the balance", async function () { const
{vendingMachine,owner,buyer, sodaVendor} = await
loadFixture(deployVendingFixture); const price = await
sodaVendor.getPrice(); await
vendingMachine.connect(buyer).buySoda({value:price}); await
vendingMachine.connect(owner).withdraw(); const bal = await
vendingMachine.getBalance(); expect(bal).to.equal(0); })
```

But here we didn’t check the balance of owner, is it even increased or not

```
it("Proving Owner withdraw the balance", async function () { const
{vendingMachine,owner,buyer, sodaVendor} = await
loadFixture(deployVendingFixture); const price = await
sodaVendor.getPrice(); await
vendingMachine.connect(buyer).buySoda({value:price}); const
balanceBefore = await ethers.provider.getBalance(owner.address); await
vendingMachine.connect(owner).withdraw(); const balanceAfter = await
ethers.provider.getBalance(owner.address); const bal = await
vendingMachine.getBalance(); expect(bal).to.equal(0);
expect(balanceAfter).to.be.gt(balanceBefore); })
```

Now we can do this in more professional way

```
it("Should transfer the full balance to the owner upon withdrawal",
async function () { // ARRANGE const { vendingMachine, owner, buyer,
sodaVendor } = await loadFixture(deployVendingFixture); const price =
await sodaVendor.getPrice(); await
vendingMachine.connect(buyer).buySoda({ value: price }); // ACT & ASSERT
(Combined) // This single line asserts that when the owner withdraws: //
1. The vendingMachine's balance DECREASES by the 'price' amount. // 2.
The owner's balance INCREASES by the 'price' amount. await expect(
vendingMachine.connect(owner).withdraw()
).to.changeEtherBalances([vendingMachine, owner], [-price, price]); });
```

Test Case #6 Handle emit event

```
it("Should emit a SodaPurchase event with correct arguments on success",
async function () { const { vendingMachine, sodaVendor, buyer } = await
loadFixture(deployVendingFixture); const price = await
sodaVendor.getPrice(); await
expect(vendingMachine.connect(buyer).buySoda({ value: price }))
.to.emit(vendingMachine, "SodaPurchase") .withArgs(buyer.address, 1);
});
```

Test Case #7: Successful Restock

- **Goal:** Prove the owner can add to the inventory.
- **Arrange:** Deploy the contract. The initial inventory is 100.
- **Act:** Have the owner call addStock(50).
- **Assert:** Check that vendingMachine.soda() now equals 150.

Test Case #8: Failed Restock by Non-Owner

- **Goal:** Prove that only the owner can restock.
- **Arrange:** Deploy the contract.
- **Act:** Have the buyer attempt to call addStock(50).
- **Assert:** Expect the transaction to.be.revertedWith("Ownable: caller is not the owner") (or your custom message).

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; contract
Voting{ struct Party{ string name; uint vote; } struct Voter{ bool
isRegistered; uint age; bool hasVoted; } address public
ElectionCommission; Party[] public politicalParty;
mapping(address=>Voter) public Voters; constructor (){
ElectionCommission = msg.sender; } modifier onlyOwner(){
require(msg.sender==ElectionCommission,"Only Election Commission can
call this function"); _; } function registerParty(string memory
_partyName) public onlyOwner{ Party memory newParty = Party({
name:_partyName, vote:0 }); politicalParty.push(newParty); } function
registerVoter(address _voterAddress, uint _age) public onlyOwner{
require(_age>=18,"You are not eligble for vote");
require(!Voters[_voterAddress].isRegistered,"You are already
registered"); Voters[_voterAddress].isRegistered = true;
Voters[_voterAddress].age = _age; } function voting(uint _partyIndex)
public { require(Voters[msg.sender].isRegistered,"You are not
registered"); require(!Voters[msg.sender].hasVoted,"You have already
voted"); require(_partyIndex>=0 &&
_partyIndex<politicalParty.length,"Invalid Party Index");
Voters[msg.sender].hasVoted = true; politicalParty[_partyIndex].vote++;
} function getWinner() public view returns(string memory partyName){
uint winnerVoteCount = 0; uint winnerIndex = 0; for(uint
i=0;i<politicalParty.length;i++){
if(politicalParty[i].vote>winnerVoteCount){ winnerVoteCount =
politicalParty[i].vote; winnerIndex = i; } } Party storage winner =
politicalParty[winnerIndex]; return winner.name; } }
```

ERC-20 Token

The problem in the early days of Ethereum. Many projects created their own digital tokens, but every single one was a unique smart contract with different function names and different rules.

- Alice's Token might have a send_tokens() function.
- Bob's Token might have a transferTokens() function.
- An exchange (like Uniswap) that wanted to support both tokens would have to write custom code for each one. This was impossible to scale.

The Solution: A Universal "Blueprint" for Money

What if everyone in the village agreed on a single, universal **blueprint** for what it means to be a "token"?

This blueprint wouldn't define the *name* of the money (Wheat, Iron, Wood) or *how much* exists. It would only define the **basic actions** that all money must support.

This standard blueprint would say:

1. **Every token must have a way to ask: "How many tokens exist in total?"**
2. **Every token must have a way to ask: "How many tokens does this person have?"**
3. **Every token must have a single, standard way to "send tokens from my account to someone else's."**

If everyone follows this blueprint, the system becomes incredibly powerful. The market owner can now build one single accounting book that can handle Wheat, Iron, and Wood tokens, because they all have the same basic functions. The vending machine can be programmed once to accept any token that follows the standard.

ERC20 is This Universal Blueprint

ERC20 is not code. It is a standard. It is an official "menu" or **interface** that defines a set of functions that a smart contract must have to be considered a standard, tradable token.

It's the community agreeing on a common language for digital mone

Required Functions (The 6 Core Functions)

1. **totalSupply() public view returns (uint256)**
   - **Purpose:** Returns the total number of tokens that exist.
2. **balanceOf(address _owner) public view returns (uint256 balance)**
   - **Purpose:** Returns the token balance of a specific address (_owner).
3. **transfer(address _to, uint256 _value) public returns (bool success)**
   - **Purpose:** Transfers _value amount of tokens from the caller's (msg.sender's) account to the _to address.
   - **MUST** fire a Transfer event.
   - Should return true on success.
4. **transferFrom(address _from, address _to, uint256 _value) public returns (bool success)**
   - **Purpose:** Transfers _value amount of tokens from the _from address to the _to address.
   - This is the function that a "spender" (like Uniswap) calls after they have been approved.
   - The caller (msg.sender) must have a sufficient allowance from the _from address.
   - **MUST** fire a Transfer event.
   - Should return true on success.
5. **approve(address _spender, uint256 _value) public returns (bool success)**
   - **Purpose:** The token owner (msg.sender) gives the _spender permission to withdraw up to _value amount of tokens.
   - **MUST** fire an Approval event.
   - Should return true on success.
6. **allowance(address _owner, address _spender) public view returns (uint256 remaining)**
   - **Purpose:** Returns the amount that the _spender is still allowed to withdraw from the _owner.

Required Events (The 2 Core Events)

Your contract **MUST** also define and emit these two events at the appropriate times.

1. **Transfer(address indexed _from, address indexed _to, uint256 _value)**
    - **When to Emit:** This event **MUST** be emitted when tokens are transferred, including during the initial creation of tokens ("minting," where _from is the zero address). It is emitted by both transfer and transferFrom.
2. **Approval(address indexed _owner, address indexed _spender, uint256 _value)**
    - **When to Emit:** This event **MUST** be emitted whenever the approve function is successfully called.

Link: https://eips.ethereum.org/EIPS/eip-20

Link: **contracts/token/ERC20/ERC20.sol** OpenZeppelin/openzeppelin-contracts

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; contract
cryptoCurrency{ address public owner; uint public totalSupply; mapping
(address=>uint) public balance; modifier onlyOwner{ require(msg.sender
== owner, "Not Owner"); _; } constructor(){ owner = msg.sender;
balance[msg.sender] = 10000; totalSupply=10000; } function mint(uint
amount) public onlyOwner{ balance[msg.sender]+=amount;
totalSupply+=amount; } function mintTo(uint amount, address to) public
onlyOwner{ require(amount<=balance[msg.sender],"Dont have enough
balance"); balance[msg.sender]-=amount; balance[to]+=amount;
totalSupply+=amount; } function transfer(uint amount, address to) public
{ require(amount<=balance[msg.sender],"Dont have enough balance");
balance[msg.sender]-=amount; balance[to]+=amount; } }
```

Now time to implement all the 6 functions

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; contract
cryptoCurrency{ address private owner; uint256 private _totalSupply;
mapping (address=>uint256) private balances; mapping (address => mapping
(address=>uint256)) private allowed; event Transfer(address indexed
_from, address indexed _to, uint256 _value); event Approval(address
indexed _owner, address indexed _spender, uint256 _value); string
private _name; string private _symbol; uint8 private _decimal;
constructor(){ _name= "Rohit"; _symbol = "R"; _decimal = 10; owner =
msg.sender; _totalSupply = 1000000 * (10**uint256(_decimal));
balances[owner]=_totalSupply; emit Transfer(address(0), owner,
_totalSupply); } function name() public view returns (string memory){
return _name; } function symbol() public view returns (string memory){
return _symbol; } function decimals() public view returns (uint8){
return _decimal; } function totalSupply() public view returns (uint256){
return _totalSupply; } function balanceOf(address _owner) public view
returns (uint256 balance){ return balances[_owner]; } function
transfer(address _to, uint256 _value) public returns (bool success){
require(_to != address(0), "ERC20: transfer to the zero address");
require(balances[msg.sender]>=_value); balances[msg.sender]-=_value;
balances[_to]+=_value; emit Transfer(msg.sender, _to, _value); return
true; } function transferFrom(address _from, address _to, uint256
_value) public returns (bool success){ require(balances[_from]>=_value);
require(allowed[_from][msg.sender]>=_value); balances[_from]-=_value;
balances[_to]+=_value; allowed[_from][msg.sender]-=_value; emit
Transfer(_from, _to, _value); return true; } function approve(address
_spender, uint256 _value) public returns (bool success){
allowed[msg.sender][_spender]=_value; emit Approval(msg.sender,
_spender, _value); return true; } function allowance(address _owner,
address _spender) public view returns (uint256 remaining){ return
allowed[_owner][_spender]; } }
```

Class Implemented Code

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; contract
RohitCoin{ mapping (address => uint256) private balances; address
private owner; uint private _totalSupply; string private _name; string
private _symbol; uint8 private _decimals; mapping (address =>
mapping(address=>uint256)) private allowed; event Transfer(address
indexed _from, address indexed _to, uint256 _value); event
Approval(address indexed _owner, address indexed _spender, uint256
_value); modifier onlyOwner(){ require(owner==msg.sender,"You are not
Owner"); _; } constructor(){ owner = msg.sender; _name = "RohitCoin";
_symbol = "RHT"; _decimals = 18; uint256 initialSupply = 1000 *
(10**uint256(_decimals)); _totalSupply = initialSupply;
balances[msg.sender] = initialSupply; emit Transfer(address(0),
msg.sender, initialSupply); } function name() public view returns
(string memory){ return _name; } function symbol() public view returns
(string memory){ return _symbol; } function decimals() public view
returns (uint8){ return _decimals; } function totalSupply() public view
returns (uint256){ return _totalSupply; } function balanceOf(address
_owner) public view returns (uint256 balance){ return balances[_owner];
} function mint(address _to, uint amount) public onlyOwner { require(_to
!= address(0), "Cannot mint to the zero address"); _totalSupply +=
amount; balances[_to] += amount; emit Transfer(address(0), _to, amount);
} function burn(uint amount) public { address user = msg.sender;
require(balances[user] >= amount, "Burn amount exceeds balance");
_totalSupply -= amount; balances[user] -= amount; emit Transfer(user,
address(0), amount); } function transfer(address _to, uint256 _value)
public returns (bool success){
require(balances[msg.sender]>=_value,"Insufficient balance");
require(address(0)!=_to,"Invalid receiver"); balances[msg.sender]-
=_value; balances[_to]+=_value; emit Transfer(msg.sender,_to,_value);
return true; } function transferFrom(address _from, address _to, uint256
_value) public returns (bool success){
require(balances[_from]>=_value,"Insufficient balance");
require(allowed[_from][msg.sender]>=_value,"Allowance Exceeded");
require(address(0)!=_to,"Invalid Receiver address"); balances[_from]-
=_value; balances[_to]+=_value; allowed[_from][msg.sender]-=_value; emit
Transfer(_from, _to, _value); return true; } function approve(address
_spender, uint256 _value) public returns (bool success){
require(address(0)!=_spender,"Invalid Spender");
require(balances[msg.sender]>=_value,"Insufficient balance");
allowed[msg.sender][_spender] = _value; emit Approval(msg.sender,
_spender, _value); return true; } function allowance(address _owner,
address _spender) public view returns (uint256 remaining){ return
allowed[_owner][_spender]; } }
```

Setup Hardhat Project

```
mkdir RohitCoin cd RohitCoin
```

Initialize a professional Hardhat project.

```
npx hardhat init
```

Install the OpenZeppelin Contracts Library

```
npm install @openzeppelin/contracts
```

Write Your Token Smart Contract

```
// File: contracts/RohitCoin.sol // SPDX-License-Identifier: MIT pragma solidity ^0.8.20; // Import the pre-built, audited ERC20 and Ownable contracts from the library. import "@openzeppelin/contracts/token/ERC20/ERC20.sol"; import "@openzeppelin/contracts/access/Ownable.sol"; // Our contract inherits all the functionality from ERC20 and Ownable. contract RohitCoin is ERC20, Ownable { /** * @dev The constructor runs once when the contract is deployed. * It sets the token's name, symbol, and initial owner. * Then it mints the total supply to the initial owner. */ constructor(address initialOwner) ERC20("RohitCoin", "RHT") Ownable(initialOwner) { // Mint 1,000,000 tokens. // Because the ERC20 contract from OpenZeppelin uses 18 decimals by default, // we multiply by 10**18 to get the correct value in wei. _mint(initialOwner, 1_000_000 * (10**18)); } }
```

Create the Deployment Script (Ignition)

```
// File: ignition/modules/DeployToken.js const { buildModule } = require("@nomicfoundation/hardhat-ignition/modules"); module.exports = buildModule("RohitCoinModule", (m) => { // We need to get the deployer's address to pass it to the constructor. const initialOwner = m.getAccount(0); // Deploy the RohitCoin contract, passing the owner's address to its constructor. const rohitCoin = m.contract("RohitCoin", [initialOwner]); return { rohitCoin }; });
```

Compile and Deploy!

```
npx hardhat compile
```

Inside the console, run the deployment:

```
// First, import your deployment module const Crypto =
require("./ignition/modules/DeployToken.js"); // Now, run the deployment
using the 'ignition' object that Hardhat provides in the console const {
rohitCoin } = await ignition.deploy(Crypto); // Get the address of the
newly deployed contract from the contract object const tokenAddress =
await rohitCoin.getAddress(); console.log(`RohitCoin deployed to:
${tokenAddress}`);
```

Now, interact with it:

```
// Get the owner const [owner] = await ethers.getSigners(); // Call
balanceOf on the live contract object const balance = await
rohitCoin.balanceOf(owner.address); // Format and display the result
ethers.formatEther(balance);
```

How to deploy the Contract

You send a transaction to the Ethereum network that has NO to address and contains the contract's compiled bytecode in its data field.

When the Ethereum network sees a transaction like this, it understands that it's not a simple payment. It interprets the bytecode, creates a new, unique contract address, and stores that bytecode at the new address.

So, the core problem every deployment method must solve is: **How do we create, sign, and send this special "contract creation" transaction?**

To do this, any method will require three essential ingredients:

1. **A Connection to the Network (A "Node"):** You need a way to talk to the Ethereum blockchain. This is your gateway.
2. **Your "Pen" (A Wallet/Private Key):** You need an account with some ETH to pay for gas and a private key to sign the transaction, proving you authorize it.
3. **Your "Message" (The Bytecode):** You need the compiled bytecode of your smart contract.

Method 1: The Manual Method (Using a Wallet like MetaMask)

Go to Remix

Connect your metamask

Get Sepolia from
https://cloud.google.com/application/web3/faucet/ethereum/sepolia

or https://www.alchemy.com/faucets/ethereum-sepolia

Compile the code

Deploy it by selecting metamask

Method 2: Using Hardhat

✅ **Step 1: Project Setup**

1. **Create Project Folder:**codeBash

```
mkdir RohitCoinProject cd RohitCoinProject
```

1. **Initialize Hardhat Project:**codeBash

```
npx hardhat --init
```

   - Choose Create a JavaScript project.
   - Accept all defaults by pressing Enter.
2. **Install OpenZeppelin Contracts:**codeBash

```
npm install @openzeppelin/contracts
```

3. **Install DotEnv for Security:**codeBash

```
npm install dotenv
```

✅ Step 2: Add Your Smart Contract Code

1. **Clean Up:** Delete the sample contracts/Lock.sol file.
2. **Create Contract File:** Create a new file at contracts/RohitCoin.sol.
3. **Paste Your Code:** Paste the exact RohitCoin contract code you provided into this new file.

```
// File: contracts/RohitCoin.sol // SPDX-License-Identifier: MIT pragma solidity ^0.8.20; // Import the pre-built, audited ERC20 and Ownable contracts from the library. import "@openzeppelin/contracts/token/ERC20/ERC20.sol"; import "@openzeppelin/contracts/access/Ownable.sol"; // Our contract inherits all the functionality from ERC20 and Ownable. contract RohitCoin is ERC20, Ownable { /** * @dev The constructor runs once when the contract is deployed. * It sets the token's name, symbol, and initial owner. * Then it mints the total supply to the initial owner. */ constructor(address initialOwner) ERC20("RohitCoin", "RHT") Ownable(initialOwner) { // Mint 1,000,000 tokens. // Because the ERC20 contract from OpenZeppelin uses 18 decimals by default, // we multiply by 10**18 to get the correct value in wei. _mint(initialOwner, 1_000_000 * (10**18)); } }
```

✅ Step 3: Get Your Credentials

1. **Get a Sepolia RPC URL:** Go to a service like Alchemy or Infura, create a free account, create a new "Ethereum Sepolia" App, and copy the **HTTPS URL**.
2. **Get Your Private Key:** Open MetaMask, select your account, and export your private key. **Keep this secret.**
3. **Get Sepolia ETH:** Use a faucet like sepoliafaucet.com or the Google Cloud Faucet to get free test ETH sent to your MetaMask address.

✅ Step 4: Configure Your Environment

1. **Create .env File:** In the root of your RohitCoinProject folder, create a file named .env.
2. **Add Secrets to .env:** Add your credentials to this file. It should look like this: